



Did you know?

204 million Emails are sent by internet users worldwide per minute. Now that's something to ponder upon



3D printed food

With the 3D printer now being used almost everywhere, and now for food, Cloudy with a chance of 3D printed food is quite a possibility. <http://dgit.in/3DFood>

Workshop

Adobe Muse and then compiled it using PhoneGap. The application load time was around 8 seconds on a Nexus 4. If you wish to make use of any of the native features then use the appropriate function call. The example for geolocation is given below.

```

<script type="text/
javascript" charset="utf-8"
src="cordova-2.2.0.js"></script>
<script type="text/javas-
cript" charset="utf-8">
    document.
    addEventListener("deviceready",
    onDeviceReady, false);
        function onDeviceReady() {
            navigator.geolocation.
            getCurrentPosition(onSuccess,
            onError);
                function
            onSuccess(position) {
                var ele-
            ment = document.
            getElementById('geolocation');
                element.innerHTML
            = 'Latitude: ' +
            position.coords.latitude
            + '<br />' +

            'Longitude: ' +
            position.coords.longitude
            + '<br />' +

            'Altitude: ' +
            position.coords.altitude
            + '<br />' +

            'Accuracy: ' +
            position.coords.accuracy
            + '<br />' +

            'Altitude Accuracy: ' + posi-
            tion.coords.altitudeAccuracy
            + '<br />' +

            'Heading: ' +
            position.coords.heading
            + '<br />' +

            'Speed: ' +
            position.coords.speed
            + '<br />' +

            'Timestamp: ' +
            position.timestamp
            +
            '<br />';
        }

```

```

function onError(error) {
    alert('code: ' +
    error.code + '\n' +
    'message: ' +
    error.message + '\n');
}
</script> </head>
<body>
<p
id="geolocation">Finding geolo-
cation...</p>
</body>

```

This code returns the following as the output.

There are numerous examples like these for making use of each native functionality using PhoneGap's API. Most of them can be found here. <http://dgit.in/PGapPlugin>

There are a few things you need to



Access native functions in PhoneGap

keep in mind while coding your website, starting off with the amount of external libraries. Use as few external libraries as possible to make the application smooth. Using libraries are convenient but there are a lot more functions present in each library than you need, so unless you intend to use one or two libraries efficiently your application will be cumbersome to use. You might have to sacrifice on looks since they will be rendered differently on different platforms. This is absolutely fine. Use CSS as much as possible to load media since they get rendered only when the media needs to be displayed. This way you save on memory usage. Instead try to make use of the memory you will save to pre-load pages that are more likely to be clicked on. The example code for jQuery Mobile is given below.

```

<a href="foo/bar/baz" data-
prefetch>link text</a>

```

The entire page gets loaded and you don't have to experience the 250-300 mil-lisecond delay that is inherently present in Webkit. PhoneGap apps appear slow because of this delay and "lazy-loading" is one of the ways of getting around this issue. Avoid long lists as much as possible and avoid gradients in CSS since these are cal-culated by the GPU of the device and will take up quite the processing power but that doesn't mean that you can't use gradients anywhere, optimise its usage.

Step 3 - Generate assets

While coding you will come up with var-ious elements and in order to get the work done as quickly as possible most program-mers insert a simple blank image of the proper dimensions. Once you've finished coding you need to start creating all those images. Do keep in mind the image densi-ties while doing so, otherwise you are likely to create an app that is compatible with very few devices. Or if you create just a few assets then your app will look wonderful on a few devices while it renders absolutely worse on everything else. This is why you need to generate multiple images/icons/ splash screens for screens of different den-sities. This way your app seems the same no matter which device you are viewing it on. You can find all the different icon sizes that PhoneGap need from this link <http://dgit.in/PGapIcon> and for the splash screens from this link <http://dgit.in/PGapSplash>. Or you could simply generate all of them by uploading a few images using the following tool. <http://dgit.in/IconGen>

Step 4 - Arrange the file structure

Once you have all the files ready, you need to arrange them in a certain order depending upon what all you are going to use. Take your existing project folder with the HTML files and all the included libraries and put them all together in a folder labelled "www". If you've made use of any plugins that we mentioned earlier or any at all then those plugins need to be kept in a folder labelled "plugins" which sits at the same level as "www". There are